Satellite Attitude Control Simulator Blueprint

Goal

Build a simulator that starts from **TLE + SGP4 orbit propagation** and then grows into a full **attitude dynamics / environment / sensor / actuator / control / estimation** framework.

Recommended development order

Phase 1. Orbit propagation MVP

Focus first on a reliable orbit truth pipeline.

Target features - Load TLE - Propagate orbit using SGP4 - Output ECI position and velocity - Convert ECI to ECEF - Convert ECEF to geodetic latitude / longitude / altitude - Save logs and plot orbit / ground track

Why first This becomes the base for: - IGRF magnetic field lookup - Sun vector / eclipse - GPS truth generation - LVLH / Nadir reference generation

---

Phase 2. Environment models

- Earth model / geodesy
- IGRF magnetic field
- Sun vector
- Eclipse
- Optional albedo / atmosphere / drag later

Phase 3. Attitude truth dynamics

- Quaternion kinematics
- Rigid body rotational dynamics
- Disturbance torques
- Reaction wheel and magnetorquer actuator models

Phase 4. Sensor models

- Gyro
- Magnetometer
- GPS
- Sun sensor
- IMU wrapper if needed

Phase 5. Estimation + control

- B-dot detumbling
- TRIAD / QUEST / EKF based attitude estimation
- RW PD or pointing controller
- Momentum dumping
- DOB-based augmentation if disturbance rejection becomes important

---

Project tree

```
prj_satellite_propagator/
│
├── README.md
├── requirements.txt
├── .gitignore
│
├── archive/
├── assets/
│   ├── earth/
│   ├── spacecraft/
│   │   ├── meshes/
│   │   └── stl/
│   └── tle/
│
├── config/
│   ├── sim_config.yaml
│   ├── orbit/
│   │   ├── tle_case.yaml
│   │   └── propagation.yaml
│   ├── spacecraft/
│   │   ├── bus.yaml
│   │   ├── inertia.yaml
│   │   ├── mtq.yaml
│   │   ├── rw.yaml
│   │   └── sensors.yaml
│   ├── env/
│   │   ├── igrf.yaml
│   │   ├── sun.yaml
│   │   └── albedo.yaml
│   └── scenario/
│       ├── orbit_only.yaml
│       ├── bdot_detumble.yaml
│       └── rw_nadir.yaml
│
├── docs/
│   ├── architecture.md
│   ├── coordinate_frames.md
│   ├── orbit_pipeline.md
```

```
│   ├── estimation_design.md
│   └── roadmap.md
│
├── experiments/
│   ├── logs/
│   ├── figures/
│   └── runs/
│
├── scripts/
│   ├── run_orbit_case.py
│   ├── run_bdot_case.py
│   └── visualize_orbit.py
│
├── src/
│   └── satsim/
│       ├── __init__.py
│       ├── main.py
│       │
│       ├── core/
│       │   ├── constants.py
│       │   ├── units.py
│       │   ├── time.py
│       │   ├── frames.py
│       │   ├── rotation.py
│       │   ├── transforms.py
│       │   ├── math_utils.py
│       │   └── state_types.py
│       │
│       ├── orbit/
│       │   ├── base.py
│       │   ├── tle.py
│       │   ├── orbit_state.py
│       │   ├── sgp4_propagator.py
│       │   ├── numerical_propagator.py
│       │   ├── perturbations.py
│       │   └── orbit_utils.py
│       │
│       ├── attitude/
│       │   ├── attitude_state.py
│       │   ├── kinematics.py
│       │   ├── rigid_body_dynamics.py
│       │   ├── reference_generators.py
│       │   └── disturbance_torques.py
│       │
│       ├── environment/
│       │   ├── earth/
│       │   │   ├── geodesy.py
│       │   │   └── gravity.py
│       │   ├── magnetic/
│       │   │   └── igrf_model.py
```

```
│       │   ├── sun/
│       │   │   ├── sun_ephemeris.py
│       │   │   └── eclipse.py
│       │   └── radiation/
│       │       └── albedo.py
│       │
│       ├── actuators/
│       │   ├── base.py
│       │   ├── magnetorquer.py
│       │   ├── reaction_wheel.py
│       │   └── allocator.py
│       │
│       ├── sensors/
│       │   ├── base.py
│       │   ├── noise.py
│       │   ├── gyro.py
│       │   ├── magnetometer.py
│       │   ├── gps.py
│       │   ├── sun_sensor.py
│       │   └── imu.py
│       │
│       ├── estimation/
│       │   ├── base.py
│       │   ├── triad.py
│       │   ├── quest.py
│       │   ├── attitude_ekf.py
│       │   └── disturbance_observer.py
│       │
│       ├── control/
│       │   ├── base.py
│       │   ├── bdot.py
│       │   ├── rw_pd.py
│       │   ├── nadir_pointing.py
│       │   └── momentum_dump.py
│       │
│       ├── spacecraft/
│       │   ├── bus.py
│       │   ├── mass_properties.py
│       │   ├── geometry.py
│       │   └── satellite_model.py
│       │
│       ├── sim/
│       │   ├── clock.py
│       │   ├── scenario.py
│       │   ├── integrator.py
│       │   ├── simulator.py
│       │   ├── logger.py
│       │   └── recorder.py
│       │
│       ├── visual/
```

```
│       │   ├── stl_loader.py
│       │   ├── orbit_viewer.py
│       │   └── attitude_viewer.py
│       │
│       └── io/
│           ├── config_loader.py
│           ├── tle_reader.py
│           └── result_writer.py
│
└── tests/
    ├── test_rotation.py
    ├── test_frames.py
    ├── test_sgp4.py
    ├── test_geodesy.py
    ├── test_igrf.py
    ├── test_bdot.py
    └── test_rw.py
```

Core design principles

1. Separate orbit and attitude

They evolve differently. - Orbit truth: position, velocity - Attitude truth: quaternion, angular rate - Different update rates are normal

2. Treat environment as independent truth models

The simulator should query environment models from current time and orbit state. - Earth - Magnetic field - Sun vector - Eclipse - Albedo

3. Separate controller and actuator

- Controller decides desired torque or desired dipole
- Allocator maps commands
- Actuator applies limits and dynamics

4. Estimation should not be mixed with truth

Truth and estimated state must be different objects.

Key state objects

Orbit state

```
@dataclass
class OrbitState:
```

```
    t_sec: float
    r_eci_m: np.ndarray
    v_eci_mps: np.ndarray
```

Attitude state

```
@dataclass
class AttitudeState:
    q_bi: np.ndarray
    w_b_radps: np.ndarray
```

Spacecraft mass properties

```
@dataclass
class MassProperties:
    mass_kg: float
    inertia_kgm2: np.ndarray
    com_m: np.ndarray
```

Environment snapshot

```
@dataclass
class EnvironmentState:
    B_eci_T: np.ndarray
    sun_eci: np.ndarray
    eclipse: bool
    albedo_flux: float | None = None
```

Orbit propagation module design

`orbit/tle.py`

Responsibilities: - TLE container - epoch parsing - validation

Example:

```
@dataclass
class TLERecord:
    name: str
    line1: str
    line2: str
```

`orbit/sgp4_propagator.py`

Responsibilities: - Create SGP4 satellite object from TLE - Propagate at arbitrary timestamps - Return ECI position and velocity

Example API:

```
class SGP4Propagator:
    def __init__(self, tle: TLERecord):
        ...

    def propagate(self, dt_utc: datetime) -> OrbitState:
        ...
```

`orbit/numerical_propagator.py`

Add later. Responsibilities: - Integrate `r, v` using ODE solver - 2-body first - then J2, drag, SRP later

---

Coordinate/frame package

This must be carefully documented.

Frames to support first

- ECI
- ECEF
- geodetic LLA
- body
- LVLH

`core/rotation.py`

- quaternion normalize
- quaternion multiply
- quaternion to DCM
- DCM to quaternion
- skew matrix

`core/transforms.py`

- ECI to ECEF
- ECEF to ECI
- ECEF to geodetic
- geodetic to ECEF
- ECI to LVLH
- body to inertial

Documentation rule

Write frame conventions explicitly in `docs/coordinate_frames.md`.

Environment model design

IGRF

`environment/magnetic/igrf_model.py`

Responsibilities: - input: time + position - output: magnetic field vector - may internally use geodetic lat/lon/alt

Example API:

```
class IGRFModel:
    def magnetic_field_ecef(self, dt_utc: datetime, r_ecef_m: np.ndarray) -> np.ndarray:
        ...
```

Sun model

`environment/sun/sun_ephemeris.py` - sun vector in ECI - later eclipse check can use Earth shadow geometry

Albedo

Add only after the basic orbit and attitude pipelines are stable.

Actuator model design

Magnetorquer

`actuators/magnetorquer.py`

Config:

```
@dataclass
class MagnetorquerConfig:
    axis_body: np.ndarray
    max_dipole_Am2: float
    time_constant_s: float = 0.0
```

Runtime: - command dipole - saturation - actual dipole - torque = m x B

Reaction wheel

`actuators/reaction_wheel.py`

Config:

```
@dataclass
class ReactionWheelConfig:
    axis_body: np.ndarray
    max_torque_Nm: float
    max_speed_radps: float
    inertia_kgm2: float
```

Runtime: - wheel speed - wheel momentum - command torque - saturation

---

Sensor model design

Base interface

```
class SensorBase(ABC):
    @abstractmethod
    def measure(self, truth_state, env_state, t_sec: float):
        pass
```

Gyro

Need: - white noise - bias random walk - scale factor optional - update rate - saturation optional

Magnetometer

Need: - body-frame magnetic field measurement - white noise - bias - hard/soft iron later

GPS

Need: - position, velocity measurement - update rate - dropout possibility

Sun sensor

Need: - field of view - eclipse handling - measurement invalid flag

---

Estimation design: state estimation vs DOB

This is the important design choice.

A. State estimation

Use when you need to estimate attitude and angular rate from sensors.

Typical progression: 1. TRIAD / QUEST from sun + magnetometer 2. Complementary filter or Mahony-style fusion 3. EKF for gyro + magnetometer + sun sensor + GPS derived references

Recommended role: - estimate `q_hat`, `w_hat` - provide state to controller

Good starting choice: - **truth-based control first** - then **gyro + magnetometer + sun sensor based estimation** - then EKF later

B. DOB (Disturbance Observer)

Use when you already have a state controller but want to reject model uncertainty and disturbance torque.

Typical purpose: - unknown external torque estimation - friction or residual actuator mismatch compensation - improve tracking robustness

Recommended role: - estimate `tau_dist_hat` - augment RW or attitude controller

What to do first?

For this project, **state estimation should come before DOB**.

Reason: - If you do not yet have a realistic sensor-to-state path, DOB helps less. - Satellite attitude control usually needs estimated attitude first. - B-dot detumbling can start without full attitude estimation, but later pointing control needs it.

Practical roadmap

1. orbit truth
2. magnetic field truth
3. attitude truth
4. gyro + magnetometer sensor models
5. TRIAD / QUEST attitude estimate
6. EKF if needed
7. RW pointing control
8. add disturbance torques
9. then test DOB for robustness

Simple rule

- **Need attitude from sensors?** -> estimation first
- **Need robustness to uncertainty/disturbance after controller works?** -> DOB after that

---

Suggested first implementation scope

Scope for first working version

- TLE reader
- SGP4 orbit propagator
- ECI to ECEF transform
- geodetic conversion
- logging to CSV
- simple orbit plot

Output example

- time UTC
- x_eci, y_eci, z_eci
- vx_eci, vy_eci, vz_eci
- lat, lon, alt

Minimal first-code tree

```
src/satsim/
├── main.py
├── core/
│   ├── time.py
│   ├── frames.py
│   ├── rotation.py
│   └── transforms.py
├── orbit/
│   ├── tle.py
│   ├── orbit_state.py
│   └── sgp4_propagator.py
├── environment/
│   └── earth/
│       └── geodesy.py
├── io/
│   ├── tle_reader.py
│   └── result_writer.py
└── sim/
    ├── scenario.py
    └── logger.py
```

Concrete first file plan

`core/time.py`

- datetime helpers

- julian date conversion if needed
- epoch stepping helper

`core/transforms.py`

- ECI to ECEF
- ECEF to geodetic support call

`orbit/tle.py`

- TLERecord dataclass

`orbit/orbit_state.py`

- OrbitState dataclass

`orbit/sgp4_propagator.py`

- wrapper around sgp4 package

`environment/earth/geodesy.py`

- ECEF to LLA
- LLA container

`io/tle_reader.py`

- load TLE from assets/tle/

`io/result_writer.py`

- write CSV

`scripts/run_orbit_case.py`

- load config
- read TLE
- propagate for 1 orbit or 24h
- save result
- plot ground track

Suggested config examples

`config/orbit/tle_case.yaml`

```yaml
tle_file: assets/tle/iss.tle
start_utc: "2026-03-27T00:00:00"
duration_sec: 5400
step_sec: 10
```

`config/sim_config.yaml`

```
scenario: orbit_only
output_dir: experiments/runs/orbit_case_001
save_csv: true
save_plot: true
```

Final recommendation

For your simulator, the best near-term path is:

1. **Build orbit truth pipeline first** using TLE + SGP4
2. Make **frame transforms and geodetic conversion rock solid**
3. Add **IGRF magnetic field model** next
4. Then build **attitude truth + MTQ/RW actuator models**
5. For control architecture, prioritize **state estimation before DOB**
6. Use DOB after attitude estimation and baseline controller are already working

That sequence will keep the project coherent and prevent the common problem of mixing truth, measurements, and control before the geometry/time pipeline is stable.